TensorFlow, GenAI & Prompt Engineering

WEEK 3

Report for Week 3

1. Overview

Week 3 extended the deep learning skill set to TensorFlow/Keras, applied it in a larger project on a real dataset, and introduced generative AI skills: systematic prompt engineering and AI-assisted software development with an audit of generated code.

Task	Title	Level	Deliverable
3.1	TensorFlow/Keras Model	Intermediate	TensorFlow notebook + comparison notes
3.2	Computer Vision / Text Classification Project (PROJECT)	Advanced	Working notebook + trained model file
3.3	Prompt Engineering Portfolio	Intermediate	Prompt evaluation sheet with iterations and outcomes
3.4	AI-Assisted Coding Task	Advanced	Working application + short audit notes

2. Task-wise Details

3.1 TensorFlow/Keras Model

INTERMEDIATE

Task Description

Build and train a TensorFlow/Keras neural network; compare it with the earlier PyTorch model.

Objective

Learn the Keras workflow and understand the differences between the two frameworks.

Approach and Methodology

- Built the network with the Keras Sequential or Functional API.
- Compiled with optimizer, loss and metrics, then trained using model.fit with validation data and callbacks such as EarlyStopping.
- Evaluated with model.evaluate and plotted training history.
- Compared with the PyTorch model on code length, flexibility, debugging experience, training time and accuracy.

Tools and Libraries

TensorFlow, Keras, Matplotlib.

Outcome and Results

The Keras model was trained and compared with the PyTorch version. *[Add: your actual result, e.g. dataset name, metric values, screenshots reference]*

Key Learnings

- Keras is faster for prototyping; PyTorch gives finer control of the training loop.
- The same architecture can reach similar accuracy in either framework.

DELIVERABLE TensorFlow notebook + comparison notes.

3.2 Computer Vision / Text Classification Project

ADVANCED | PROJECT

Task Description

Implement a TensorFlow computer-vision or text-classification project on a real dataset.

Objective

Deliver an end-to-end deep learning project from raw data to a saved, evaluated model.

Approach and Methodology

- Selected a real dataset and defined the problem and success metric.
- Preprocessed data: image resizing/augmentation, or text cleaning, tokenisation and padding/embeddings.
- Designed and trained the model (CNN or transfer learning for vision; embeddings with LSTM/dense layers for text).
- Tuned the model, evaluated on a test set with precision, recall, F1 and a confusion matrix, and analysed errors.
- Saved the trained model file and documented the notebook.

Tools and Libraries

TensorFlow/Keras, NumPy, Pandas, Matplotlib, Scikit-learn metrics.

Outcome and Results

The project was completed with a trained and saved model. *[Add: your actual result, e.g. dataset name, metric values, screenshots reference]*

Key Learnings

- Real data needs more preprocessing than benchmark data.
- Transfer learning and embeddings improve results with limited data.

DELIVERABLE Working notebook + trained model file.

3.3 Prompt Engineering Portfolio

INTERMEDIATE

Task Description

Design and test LLM prompts for reasoning, extraction, summarization, and coding.

Objective

Learn to design prompts that give reliable, high-quality LLM outputs and to evaluate them systematically.

Approach and Methodology

- Defined test cases for four categories: reasoning, information extraction, summarisation and coding.
- Wrote an initial prompt for each and then iterated using role assignment, clear instructions, output format constraints, few-shot examples and step-by-step reasoning.
- Scored outputs on accuracy, completeness, format adherence and consistency.
- Recorded every iteration and outcome in an evaluation sheet.

Tools and Libraries

LLM chat interface/API, spreadsheet for evaluation.

Outcome and Results

The portfolio shows measurable improvement across iterations in each category. *[Add: your actual result, e.g. dataset name, metric values, screenshots reference]*

Key Learnings

- Specific, structured prompts with examples are far more reliable.
- Prompts should be tested like code, with cases and scores.

DELIVERABLE Prompt evaluation sheet with iterations and outcomes.

3.4 AI-Assisted Coding Task

ADVANCED

Task Description

Build a small app with an AI coding assistant; review, correct, and audit the generated code.

Objective

Use AI to speed up development while keeping human responsibility for correctness and quality.

Approach and Methodology

- Specified the app requirements and generated the code with an AI assistant in small steps.
- Reviewed the code for logic errors, edge cases, security issues (input validation, secrets) and style.
- Corrected issues, added tests and ran the application.
- Wrote short audit notes listing issues found, fixes applied and reliability observations.

Tools and Libraries

AI coding assistant, Python, Git.

Outcome and Results

A working application was delivered with an audit of the AI-generated code. *[Add: your actual result, e.g. dataset name, metric values, screenshots reference]*

Key Learnings

- AI-generated code must always be reviewed and tested.
- Small, well-specified requests give better code.

DELIVERABLE Working application + short audit notes.

3. Summary of the Week

- Gained working skill in both PyTorch and TensorFlow/Keras.
- Delivered an end-to-end deep learning project.
- Built a tested prompt engineering portfolio.
- Practised responsible AI-assisted development with a code audit.

4. Challenges and How They Were Handled

- Inconsistent LLM outputs; improved with format constraints and few-shot examples.
- AI code contained subtle bugs; caught through review and tests.

5. Outlook

Week 4 focuses on LangChain, LangGraph and AI agents, culminating in the capstone AI agent system.